
Weike Liu

| RL, MicroDuck, MuJoCo

目录

- 1 安装 mujoco
- 2 第一个 Hello World
- 3 看一些真家伙
- 4 MuJoCo 是什么？
- 5 做一个机械臂
 - 5.1 第一步：定义机器人模型（MJCF/XML 文件）
 - 5.2 第二步：参数配置文件（YAML）
 - 5.3 第三步：主 Python 程序（完整开发流程）
 - 5.3.1 导入依赖与初始化
 - 5.3.2 加载 YAML 配置（`load\_config` 函数）
 - 5.3.3 加载 MuJoCo 模型
 - 5.3.4 进入仿真主循环（带被动查看器）
 - 5.3.5 主循环内部（`while viewer.is\_running()`）
 - 5.3.6 退出循环与结束
 - 5.3.7 补充说明
 - 5.4 第四步：如何运行与效果验证
- 6 回到小鸭子
 - 6.1 MicroDuck 主仓库
 - 6.1.1 整体结构
 - 6.1.2 MicroDuck 机器人控制的核心 robotd
 - 6.1.2.1 robotd 的执行流程
 - 6.1.2.2 第一部分：robotd 主流程
 - 6.1.2.2.1 第一块：参数和配置
 - 6.1.2.2.2 第二块：robotd init 是一条特殊路径
 - 6.1.2.2.3 第三块：抢占 robotd 的 Unix socket
 - 6.1.2.2.4 第四块：创建 RobotState

-
- 6.1.2.2.5 第五块：创建 Intents
 - 6.1.2.2.6 第六块：创建 poweroff 回调
 - 6.1.2.2.7 第七块：启动 control thread
 - 6.1.2.2.8 第八块：主线程启动 IPC server
 - 6.1.2.2.9 IPCserver 本身怎么走？
 - 6.1.2.2.10 IPChandler 做什么？
 - 6.1.2.2.11 主线程什么时候退出？

6.1.2.3 第二条主线：50Hz Control Loop

- 6.1.2.3.1 Control Loop 初始化阶段
- 6.1.2.3.2 第一块：确定当前 policy
- 6.1.2.3.3 第二块：创建 Safety
- 6.1.2.3.4 第三块：读取启动姿态
- 6.1.2.3.5 第四块：创建 Controller
- 6.1.2.3.6 第五块：建立 50 Hz ticker
- 6.1.2.3.7 进入 while
- 6.1.2.3.8 Tick 第 1 块：等待周期
- 6.1.2.3.9 Tick 第 2 块：读取传感器
- 6.1.2.3.10 Tick 第 3 块：Coast
- 6.1.2.3.11 Tick 第 4 块：更新 IMU / odometry / fall 状态
- 6.1.2.3.12 Tick 第 5 块：读取 Intent Snapshot
- 6.1.2.3.13 Tick 第 6 块：Safety Gate
- 6.1.2.3.14 Tick 第 7 块：处理一次性 power request
- 6.1.2.3.15 Tick 第 8 块：处理 Skill / Sit / Ground Pick
- 6.1.2.3.16 Tick 第 9 块：处理声音
- 6.1.2.3.17 Tick 第 10 块：处理 Mode Switch / Policy Reload
- 6.1.2.3.18 Tick 第 11 块：Shutdown / Battery Shutdown
- 6.1.2.3.19 Tick 第 12 块：Limp-Fall
- 6.1.2.3.20 Tick 第 13 块：决定是否进入 driving

-
- 6.1.2.3.21 Tick 第 14 块: 处理 driving 开始 / 停止
 - 6.1.2.3.22 Tick 第 15 块: 电池电压补偿
 - 6.1.2.3.23 Tick 第 16 块: 生成 JointTargets
 - 6.1.2.3.24 Tick 第 17 块: Controller.step()
 - 6.1.2.3.25 Tick 第 18 块: Theremin/Chorale/Mouth
 - 6.1.2.3.26 Tick 第 19 块: 最终进入 Safety
 - 6.1.2.3.27 Tick 第 20 块: 发布 RobotState
 - 6.1.2.3.28 Tick 第 21 块: 记录 loop health

6.1.2.4 robotd 的一些主要设计要点

6.2 robotd 中关于 ONNX 的模型声明、例化、调用

- 6.2.1 定义: 定义一个 ONNX 模型对象
- 6.2.2 创建: 把 '.onnx' 文件加载成 'Session'
- 6.2.3 'robotd' 创建它
- 6.2.4 执行: 调用 ONNX
- 6.2.5 'robotd' 是怎么调用它的?
- 6.2.6 三个动作总结

6.3 从 ONNX 文件到程序输出: 完整流程

- 6.3.1 先理解: ONNX 文件本身是什么
- 6.3.2 完整过程 (四步)

7 MicroDuck 的训练

8 浏览器仿真工具 (可视化理解架构运行)

9 官方文档与特性说明 (补充架构设计理念)

10 RL 算法

1 安装 MUJOCO

在 Windows 11 上, MuJoCo 早已是纯 Python 包, 不需要申请许可证或配置复杂环境变量。

打开命令提示符(cmd)或 PowerShell。

执行安装命令 `bash: pip install mujoco`

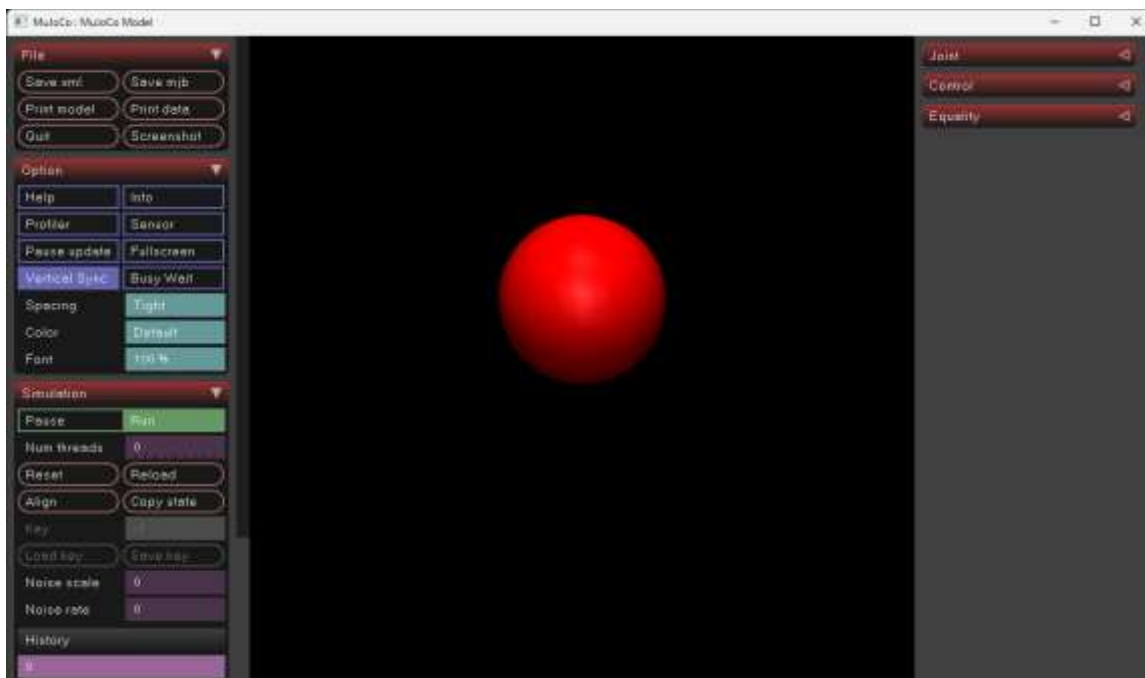
这个命令会自动装好核心库和渲染依赖。

安装后可用 `python -c "import mujoco; print(mujoco.__version__)"` 验证。

2 第一个 HELLO WORLD

[HelloWorld.py](#)

运行: `python HelloWorld.py`, 立即出现:



3 看一些真家伙

安装 `yaml`、`torch` 等等。

`Pip install pyyaml`

`Pip install torch`

```
git clone https://github.com/limxdynamics/humanoid-description.git
```

进入目录，然后：

```
python -m mujoco.viewer --mjcf=HU_D04_description/xml/HU_D04_01.xml
```

此外，还有宇树机器人：git clone <https://github.com/yueruyin/robot-unitree-g1.git>

4 MuJoCo 是什么？

MuJoCo 的核心是一个用 C 语言写成的物理引擎库，Python 只是其中最流行、最方便的一种调用方式。

具体来说，你有以下 4 种使用途径：

- ◆ **Python 接口（最常用的）**：通过 `pip install mujoco` 安装。这是官方维护的 Python 绑定，把 C 库包装成了 Python 模块。你现在运行脚本、查看模型用的都是这种方式。
- ◆ **C/C++ 原生调用**：如果你是做机器人底层开发或嵌入式的，可以直接下载 MuJoCo 的 C 库（.dll/.so/.dylib），在 C/C++ 代码里直接调用它的物理计算函数。
- ◆ **命令行工具**：安装 Python 包后，会自动附带命令行工具。比如你之前用过的 `python -m mujoco.viewer --mjcf=xxx.xml`，本质上是调用了内置的可视化程序。
- ◆ **其他语言的绑定**：社区还提供了 Julia（MuJoCo.jl）、Rust（mujoco-rs）甚至 MATLAB 的接口，覆盖了大部分主流编程语言。

总结：Python 是入门和做强化学习（RL）的首选，但 MuJoCo 本身非常“通用”，不绑定任何语言。

1. 核心定位：库（Library）vs. 平台（Platform）

MuJoCo 是“库”：它就像 OpenCV（计算机视觉库）或 FFmpeg（音视频库）。它是一套底层计算工具，没有“主界面”。你的代码是“总司令”，调用它的 API 来计算物理、接触力、关节角度，然后你把计算结果用别的工具画出来。它完全围绕代码工作。

Isaac Sim 是“平台/应用”：它就像 Unity 或 Unreal 引擎。它是一个巨大的图形化集成环境。你绝大部分时间在 GUI 里拖拽模型、搭建场景、设置光源，点按钮运行仿真。代码（Python）只是用来辅助 GUI 操作或编写复杂逻辑的“插件”。

2. 关于 GUI 的“陷阱”（很多人误解的地方）

你可能会问：“可是我用 `python -m mujoco.viewer` 不是也弹出一个窗口（GUI）了吗？”

注意区分：MuJoCo 自带的这个窗口，本质上是一个“调试监视器”，而不是“开发平台”。

Isaac Sim 的 GUI：你是在它里面工作（建场景、改物理参数、看渲染效果）。

MuJoCo 的 viewer：你只是把它当显示器（用来看看机器人到底跑得怎么样，腿有没有抖，有没有摔倒）。一旦你关掉这个 viewer，你的 Python 脚本依然可以在后台计算物理数据（比如记录日志或训练 AI），而 Isaac Sim 一旦关掉 GUI，整个仿真环境就结束了。

5 做一个机械臂

5.1 第一步：定义机器人模型（MJCF/XML 文件）

MuJoCo 使用 MJCF（MuJoCo XML）格式描述机器人。这个文件定义了机械臂的几何形状、关节、质量、颜色等。建一个 [two link arm.xml](#) 文件。

5.2 第二步：参数配置文件（YAML）

YAML 在这里负责存放“控制算法的参数”，避免写死在 Python 代码里：[config.yaml](#)。

5.3 第三步：主 PYTHON 程序（完整开发流程）

[run_arm.py](#)，这是核心代码。把加载模型、读取 YAML、控制循环、可视化全部整合在一起。

程序有很好的注释，非常易读。

这个 Python 程序（`run\_arm.py`）实现了一个基于 MuJoCo 物理引擎的两连杆机械臂轨迹跟踪仿真。它通过 YAML 配置文件设定关节目标轨迹，利用 XML 模型中内置的 `position` 执行器（自带 PD 控制）直接下发期望角度，从而驱动机械臂运动。下面按执行顺序逐步讲解其流程：

5.3.1 导入依赖与初始化

导入 `time`、`numpy`、`yaml`、`mujoco` 及其 `viewer` 子模块。这些库分别用于时间控制、数值计算、配置文件解析、物理模拟和可视化。

5.3.2 加载 YAML 配置（`load\_config` 函数）

尝试打开 `config.yaml` 文件并解析为字典。

- 若文件不存在，则使用默认参数（仅包含轨迹幅值、频率和相位差），并给出警告提示。

- 配置内容用于后续生成两个关节的目标角度轨迹。

5.3.3 加载 MuJoCo 模型

通过 `mujoco.MjModel.from\_xml\_path("two\_link\_arm.xml")` 读取 XML 模型文件。

若文件加载失败，程序打印错误并退出。

接着创建 `MjData` 对象，用于存储仿真状态（位置、速度、控制量等）。

5.3.4 进入仿真主循环（带被动查看器）

使用 `mujoco.viewer.launch\_passive()` 启动一个被动（非阻塞）可视化窗口，并传入模型和数据。

记录起始时间 `start\_time`，用于计算仿真经过的时间 `elapsed`。

5.3.5 主循环内部（`while viewer.is\_running()`）

每次迭代执行以下步骤：

a. 计算期望关节角度

- 根据配置中的 `amplitude1`、`freq1` 等参数，利用正弦函数生成两个关节的目标角度：

- $\text{target1} = A1 * \sin(2\pi * f1 * t)$

- $\text{target2} = A2 * \sin(2\pi * f2 * t + \phi2)$

- 这里 `t` 为仿真运行时间（物理时间，非 CPU 时间）。

b. 直接下发目标角度

- 将计算出的 `target1` 和 `target2` 赋值给 `data.ctrl[0]` 和 `data.ctrl[1]`。

- 关键点：XML 模型中关节执行器类型为 ``，其内部已经定义了 `kp`（比例增益）和 `kv`（速度增益），因此 MuJoCo 会自动将角度误差转化为力矩，**无需在 Python 中手动计算 PD 控制律**。

c. 步进物理引擎

- 调用 `mujoco.mj\_step(model, data)` 推进仿真一步，更新位置、速度、力矩等所有状态。

d. 同步可视化

- 调用 `viewer.sync()` 将当前数据渲染到显示窗口。

e. 控制仿真速度

- 使用 `time.sleep(model.opt.timestep)` 让程序休眠一个物理步长的时间（通常为几毫秒），从而保证仿真运行速度与真实时间大致对齐（实时仿真），避免循环过快。

5.3.6 退出循环与结束

- 当用户关闭查看器窗口或按下停止键时，`viewer.is_running()` 变为 `False`，循环终止。

- 打印 “仿真结束”，程序退出。

5.3.7 补充说明

- 整个流程是 ** “配置 → 模型加载 → 实时轨迹生成 → 直接位置控制 → 物理模拟 → 渲染” ** 的闭环。

- 该设计将底层控制（PD 参数）置于 XML 中，上层 Python 只负责轨迹规划，实现了控制逻辑与物理仿真的分离，便于调试和参数调整。

- 仿真的实时性依赖于 `time.sleep`，但实际物理步长由 `model.opt.timestep` 决定（通常固定），而 `sleep` 仅用于节流，若计算机性能不足可能导致仿真变慢，但不会影响物理精度。

5.4 第四步：如何运行与效果验证

执行脚本： `python run_arm.py`

看到：一个绿色+蓝色的机械臂，底座旋转（关节 1），同时大臂上下摆动（关节 2），末端橙色小球画着优雅的“8”字轨迹。你可以用鼠标拖拽旋转视角，右键平移，滚轮缩放。

透过这个例子，理解 MuJoCo 开发的“四步法”

环节	文件/代码	作用解析
① 模型定义	*.xml (MJCF)	这是机器人的“物理蓝图”。定义了几何形状、质量、关节类型（hinge/ball/free）、摩擦、颜色等。MuJoCo 的精华全在 XML 的调参里。
② 参数配置	*.yaml	这是“控制算法的调参表”。把频率、幅度、增益拿出来，方便做实验时反复修改，避免频繁改 Python 代码。
③ 核心逻辑	Python 脚本	负责加载（模型+YAML）、计算（目标轨迹）、执行（赋值

		data.ctrl)、步进 (mj_step)。这是你写算法的战场。
④ 可视化循环	viewer.launch_passive	被动循环让你掌握主动权——先算控制，再步进物理，最后刷新画面。这比 launch（全自动）更适合做算法开发。

进阶小贴士

调整参数：修改 config.yaml 里的 amplitude 或 freq，保存后重新运行，观察机械臂摆动变化，无需改 Python。

导出数据：在循环里加一行 `print(data.body('tip').xpos)`，就能实时打印末端执行器的三维空间位置，用于分析轨迹。

改成关节空间跟随：如果你想让他保持某个固定姿势，把 `target1 = 0.5` 写死就行了。

这个框架几乎涵盖了所有 MuJoCo 仿真项目的基础结构。你可以把 `two_link_arm.xml` 替换成 `Unitree G1` 的模型，把正弦波替换成强化学习策略，就升级成进阶项目了。

6 回到小鸭子

如果对 ONNX 等内容不够了解，可以先看 6.3。

MicroDuck 有这样的一些官方开源仓库：

- 官方主仓库，Microduck 的核心仓库，包含 SDK 和机器人本体软件。

地址：<https://github.com/pollen-robotics/microduck/tree/main>

开源协议：Apache-2.0，这意味着你可以自由地使用、修改、复制和分发代码。包括：

商业使用：可以将代码用在商业软件或内部项目中。

闭源分发：将代码集成到你自己的产品中，无需开源你自己的源代码。

再许可：可以将整个作品（包含 Apache-2.0 代码）以其他协议分发。

你需要遵守的义务：

保留声明：分发代码时，必须保留原始代码中的版权声明、许可声明和 NOTICE 文件中的所有 attribution 信息。

标记修改：如果你修改了文件，需要明确标注出修改的地方。

附带协议：再分发时，需要附上一份 Apache-2.0 的协议副本。

-
- 官方强化学习仓库，这个仓库专注于强化学习与仿真到现实（sim-to-real）的部署工具。地址：``https://github.com/pollen-robotics/microduck_rl``
 - 官方相关工具

Microduck Studio：一个本地的 Web 开发工作室，用于调试、可视化和编排 Microduck 的工作流。地址：``https://github.com/microai-lab/microduck-studio``

- 此外，还有第三方复刻项目，是一个由社区发起的第三方复刻项目，旨在从官方公开的仿真模型和源码中还原出装配图、CAD 装配体等。地址：``https://github.com/fanhao375/microduck-replica``

需要注意的是，Microduck 的软件是开源的（Apache-2.0），但硬件本身并不开源。

6.1 MICRODuck 主仓库

地址：<https://github.com/pollen-robotics/microduck/tree/main>。这个仓库里面是运行在 MicroDuck 里面的 RK3566 处理器上的软件，MicroDuck 有时候把它叫做“runtime”运行时软件。

Microduck 不是一个“单一程序”，而是一套运行在机器人 RK3566 Linux 板子上的 Rust 多进程机器人操作系统/运行时。

其中核心是 **robotd**，负责真正的机器人闭环控制；其他 **daemon**^[1] 围绕它分别负责配置、蓝牙、手柄、摄像头/远程通信、ToF、软件更新。所有控制类服务通过 Unix Socket + JSON-RPC 2.0/NDJSON 通信。

1 在 Linux 中，**daemon**（守护进程）就是一直在后台运行、默默提供服务、没有交互界面的特殊进程。你可以把它理解为电脑里的“幕后工作人员”或“系统服务员”：你不需要直接跟它说话（没有图形界面或命令行交互）；它随时待命（一直在内存中运行）；有请求时它就工作（响应客户端的调用）；没有请求时它就闲着（占用极少的 CPU 资源）。一个容易混淆的点：

daemon（守护进程）：后台运行，无交互，提供系统服务。

而 **service**（服务）：更偏“功能层面”的叫法。守护进程是服务的具体实现进程，两者常混用。比如“SSH 服务”由 **sshd** 这个守护进程提供。

如何创建自己的守护进程（简要），如果你用 C 语言编写，标准步骤是：

fork() 创建子进程，父进程退出；子进程调用 **setsid()** 创建新会话；关闭文件描述符（标准输入/输出/错误）；切换工作目录到 **/**；设置文件权限掩码 **umask(0)** 等等。

不过现在更推荐直接编写 **systemd service** 单元文件，让 **systemd** 帮你管理，省去这些底层细节。

6.1.1 整体结构

仓库本身是一个 Rust workspace，目前 Cargo workspace^[2]有 19 个 member，包括 daemon、控制算法库、CLI、IPC 协议、工具等。

它有这些“入口”：

- Linux/systemd 服务入口

机器人启动后真正常驻运行的程序主要是：

入口	类型	核心作用
robotd	daemon	机器人运动控制核心
configd	daemon	WiFi、机器人身份、配对、系统配置
updaterd	daemon	软件/模型更新、验证、回滚
btd	daemon	BLE API 入口
padd	daemon	游戏手柄入口
mediad	daemon	摄像头/麦克风/WebRTC/远程 API
tofd	daemon	ToF 深度传感器

这 7 个是整个运行时最核心的服务划分。架构文档也明确说明 systemd 是它们的 supervisor，负责生命周期、崩溃重启、启动顺序和 watchdog。

2 Cargo Workspace 是 Rust 生态中 Cargo 工具的专属专有名词，是 Rust 官方为管理多关联包项目专门设计的核心特性，在其他编程语言的构建体系中没有完全对等的原生同名概念。核心定义：

Cargo Workspace 是一组被协同开发、相互关联的 Rust 包（crate）的集合，所有成员包会共享同一份 Cargo.lock 依赖锁文件和同一个 target 编译输出目录，从根本上避免了多包开发时的依赖版本碎片化和重复编译问题。

核心特性：

统一管理多包：可以将一个大型项目拆分为多个职责单一的独立 crate，所有 crate 都在同一个工作空间下协同维护，典型案例就是 Rust 官方的 rustc 编译器仓库，整个项目就是一个巨型 Cargo Workspace，管理着近 50 个相互依赖的子 crate。

共享配置与依赖：支持在根目录的 Cargo.toml 中通过 [workspace.dependencies] 定义全局共享依赖，所有子 crate 可以直接继承这些依赖版本，无需重复编写配置，大幅降低依赖版本冲突的概率。

灵活的构建控制：既可以在根目录执行 cargo build 一次性编译所有成员包，也可以通过 -p 包名 参数单独构建、测试或发布指定的某个子 crate，兼顾整体构建效率和单包迭代的灵活性。

两种清单模式：分为虚拟清单模式（根目录的 Cargo.toml 仅包含 [workspace] 配置，本身不作为独立 crate）和根目录作为成员包的模式，适配不同的项目组织需求。

- 用户/开发者 CLI 入口

它另外有两个非常重要的 CLI（Command Line Interface）：`robotctl`、`duckctl`。

✧ **Robotctl**：这是直接在机器人 Linux 板子上执行的管理 CLI。

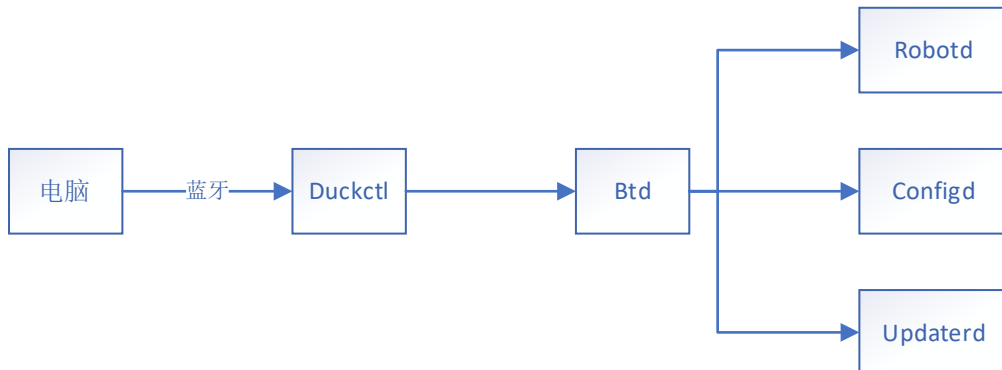
它可以有这样的一些操作操作：

```
robotctl version
robotctl health
robotctl monitor
robotctl configure
robotctl policy ...
robotctl robot ...
robotctl pad ...
robotctl update ...
robotctl wifi ...
robotctl system ...
robotctl voice ...
```

官方 `cheat sheet` 把 `robotctl` 定义成机器人上的主要控制/诊断入口。

✧ **Duckctl**，这是机器人外面的电脑上的 CLI。它通过 **Bluetooth** 找到 **Microduck**，然后发送和机器人内部相同的 **JSON-RPC** 请求。

所以：



代码注释明确写着，`duckctl` 是“`phone app` 的 `stand-in`”，只用于开发，不部署到机器人上。

6.1.2 MicroDuck 机器人控制的核心 robotd

它是真正让鸭子动起来的实时控制 daemon。robotd 执行 50Hz 控制 loop，驱动 15 个舵机，并运行神经网络 policy。robotd 是唯一能够控制电机的 daemon，其原因后面再讲。MicroDuck 用 rust 语言编写，因此许多概念涉及 rust 语言的概念，如下面的 slot、tokio 等等。

6.1.2.1 robotd 的执行流程

MicroDuck 的 robotd 是用 rust 语言写的，它的入口是：

```
#[tokio::main]
```

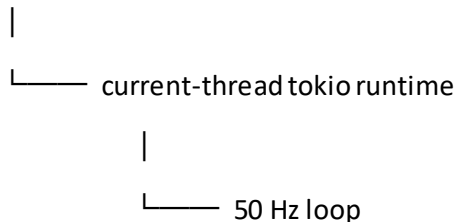
```
async fn main() -> ExitCode
```

程序启动后，main()

1. 读取 robotd.toml
2. 创建/run/robotd.sock
3. 创建 RobotState + Intents
4. 启动 control thread
5. 当前主 tokio runtime 跑 IPC server

非常值得注意的一点：第 4 步启动的 Controlloop 是一个线程，不是和第 5 步 IPC 共用一个 runtime。代码专门创建了一个：

OS thread: control



原因是 Dynamixel serial I/O 是 blocking（阻塞）的。如果把 control loop 和 IPC 放在普通 tokio runtime：

IPC request



tokio worker



blocking serial read（阻塞式的串口读）



整个 worker 被占住，IPC 循环中的其他任务就可能影响控制周期。

因此把 main.rs 看成一个“主流程 + 控制循环”，重点只抓两个主线：

1. robotd 主流程：进程启动后，怎样建立服务、创建共享状态、启动控制线程、提供 IPC。
2. 50 Hz control loop：控制线程里面每一个 tick 按什么顺序执行。

下面以当前 main 分支代码为准。robotd/src/main.rs 当前约 7600 行，但真正决定运行时主流程的核心区域集中在 main()、spawn_control_thread() 和 control_loop()。

6.1.2.2 第一部分：robotd 主流程

main() 是真正的进程入口。代码从：

```
#[tokio::main]
```

async fn main() -> ExitCode 开始。

这里实际上做的是一系列初始化和组装，而不是控制机器人。按代码顺序：

main()

- └─ 1. 解析命令行参数
- └─ 2. 初始化 logging
- └─ 3. 加载 robotd.toml
- └─ 4. 处理 --port / --no-policy
- └─ 5. 如果是 `robotd init`，走独立 init 流程
- └─ 6. claim Unix socket
- └─ 7. 创建 RobotState
- └─ 8. 创建 Intents
- └─ 9. 创建 poweroff callback
- └─ 10. 创建 control thread
- └─ 11. 启动 IPC server
- └─ 12. 等待 server / SIGTERM

所以 main() 本质上是把 robotd 的运行环境组装起来，然后把两个主要执行实体启动起来：control thread + IPC server。

6.1.2.2.1 第一块：参数和配置

Params 是静态配置。比如：bus port、control period、policy、safety、deadman、gain、limp fall、audio、mode 等等。而后面运行过程中不断变化的东西，不放在 Params

里，而放到：`RobotState`、`Intents`、`Controller` 这三个不同层次里。

6.1.2.2.2 第二块：`robotd init` 是一条特殊路径

`robotd init` 是一条 CLI 命令，它的执行是一条单独的路径，完成初始化后，机器人是“建立机器人已知姿态”，而不是正常的持续控制服务。

6.1.2.2.3 第三块：抢占 `robotd` 的 Unix socket

然后才允许启动后面的 `control thread`。代码明确要求在启动可能接触电机的线程之前先拥有 `endpoint`。

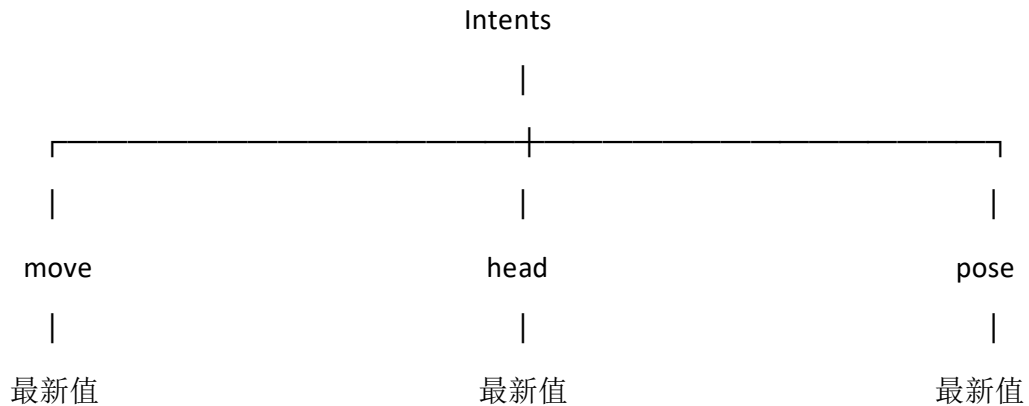
6.1.2.2.4 第四块：创建 `RobotState`

接下来：`let state = Arc::new(RobotState::new(...));`这里的 `RobotState` 很重要。它不是“机器人当前所有状态的 `Mutex`”。它主要是：`control loop` → `IPC` 的状态发布通道。

6.1.2.2.5 第五块：创建 `Intents`

接下来：`let intents = Arc::new(Intents::new());`

这个对象是 `IPC` 和 `control loop` 之间的命令交换区。注意，它不是普通的 `command queue`。更像：



比如客户端不断发送：

```
robot.move(vx=0.1)
```

```
robot.move(vx=0.2)
```

```
robot.move(vx=0.3)
```

`control loop` 并不是逐条消费：

```
0.1
```

```
0.2
```

```
0.3
```

而是在 tick 时：

```
let snapshot = intents.snapshot();
```

拿到当前这一时刻的命令快照。

所以：

IPC 产生 command



Intents



20ms 时刻 snapshot



当前控制周期使用

这是一个非常关键的架构点。

6.1.2.2.6 第六块：创建 poweroff 回调

然后：let poweroff: PowerOff = Arc::new(...)。它本身不参与普通控制。它只是给 control loop 一个：“如果决定关机，真正执行 systemctl poweroff”的能力。

所以 control loop 不直接依赖 shell/systemd 逻辑，而是收到一个：PowerOff callback。

这也方便测试。

6.1.2.2.7 第七块：启动 control thread

然后：let control = spawn_control_thread(...)这是整个 main() 的分界点。到这里：

main thread



├── RobotState

├── Intents

├── socket



└── spawn_control_thread()

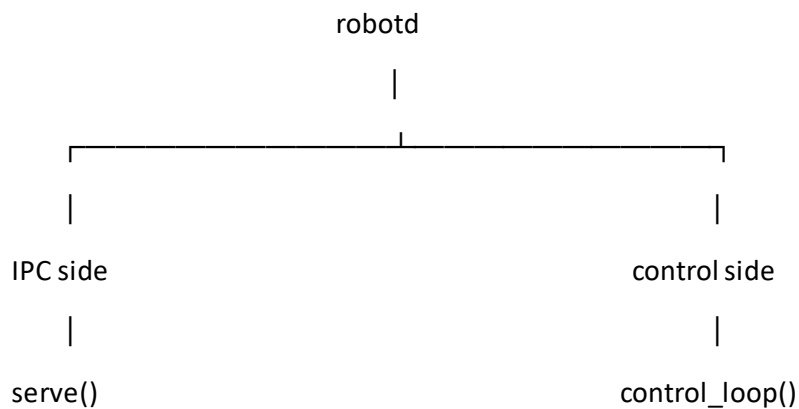


control thread

注意：真正的机器人控制从这里开始进入另一个执行上下文。

6.1.2.2.8 第八块：主线程启动 IPC server

`control thread` 启动后，主线程继续：`let serving=serve(...)`。也就是说：



然后：

```
tokio::select! {  
    result = serving => ...  
    _ = shutdown() => ...  
}
```

主线程就在这里等待。

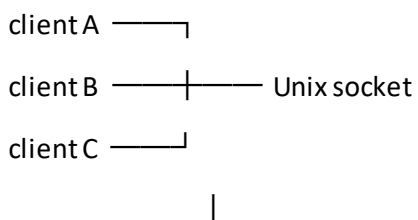
6.1.2.2.9 IPCserver 本身怎么走？

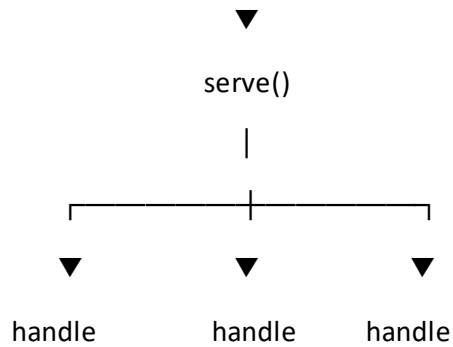
`serve()` 很简单：

loop

```
|—— listener.accept()  
|—— 得到 UnixStream  
|—— tokio::spawn(handle(...))
```

也就是说：





每个 client 都可以独立处理。代码就是 `accept` → `clone state/intents` → `tokio::spawn(handle)`，即每个 client 都由各自的 `handle` 来处理。

6.1.2.2.10 IPC handler 做什么？

`handle()` 收到 JSON-RPC/NDJSON 请求。它有两类事情：

第一类：查询

例如：

`robot.health`

`robot.state`

`robot.policies`

`robot.safeToRestart`

这些主要读：RobotState

第二类：控制

例如：

`robot.move`

`robot.head`

`robot.init`

`robot.relax`

`robot.do`

`robot.setMode`

...

这些不是直接控制电机。而是由 `request` 产生 `intent` 去 `control thread`。因此最终都要等 `control loop` 的下一 `tick`。

6.1.2.2.11 主线程什么时候退出？

当：SIGTERM 或者：IPCserver 出错，主流程结束。然后：

```
state.shutdown.store(true, ...)
```

通知 control loop：“停止。”，然后：

```
control.join()
```

等待 control thread 真正退出。

最后：

```
remove_file(socket)删除 Unix socket。
```

这里还有一个非常重要的细节：不是直接 kill control loop。代码要求：

```
shutdown = true
```

↓

```
control loop 自己结束当前 tick
```

↓

```
退出
```

↓

```
join
```

原因是不能在 Dynamixel bus transaction 中间强行 abort。

6.1.2.3 第二条主线：50 Hz Control Loop

这才是 robotd 的核心。入口：async fn control_loop<T: RobotIo>(...)，代码从约 1600 行开始。它启动以后不是马上：

```
while true
```

```
    policy
```

而是先建立控制环境。

6.1.2.3.1 ControlLoop 初始化阶段

按照代码顺序：

```
control_loop()
```

```
└─ 1. 准备 policy 配置
```

```
└─ 2. 清理不可加载的 override
```

```
└─ 3. resolve policy
```

-
- └─ 4. 创建 Safety
 - └─ 5. 读取并采用 startup pose
 - └─ 6. 创建 Controller
 - └─ 7. 创建 50Hz ticker
 - └─ 8. 初始化控制循环状态

6.1.2.3.2 第一块：确定当前 policy

6.1.2.3.3 第二块：创建 Safety

6.1.2.3.4 第三块：读取启动姿态

6.1.2.3.5 第四块：创建 Controller

6.1.2.3.6 第五块：建立 50 Hz ticker

然后：`let mut ticker = tokio::time::interval(period);`正常情况下：`period = 20ms`，然后：

`ticker.set_missed_tick_behavior(`

`MissedTickBehavior::Skip`

`);`

这点非常重要。它意味着：

tick 1

tick 2

tick 3 ← 这次太慢，错过

tick 4

不会补：

tick 3

tick 3

而是直接进入最新时间点。控制系统不希望堆积旧 `command`。代码也明确说明选择 `Skip` 是为了避免 `backlog` 和时间漂移。

6.1.2.3.7 进入 while

代码：

```
while !state.shutdown.load(...) {
```

```
    ticker.tick().await;
```

这就是机器人真正的：50Hz 主循环。下面一定要严格按照代码顺序理解。

6.1.2.3.8 Tick 第 1 块：等待周期

每次：`ticker.tick().await`; 等到下一个控制周期。然后：`let tick_start=Instant::now();`记录本 tick 起点。

6.1.2.3.9 Tick 第 2 块：读取传感器

6.1.2.3.10 Tick 第 3 块：Coast

如果读传感器失败。代码：`let sensors=coast.sample(fresh);`含义：

fresh read 成功



使用 fresh

fresh read 失败



短时间使用 last good sample

最多 `COAST_TICKS=3` 也就是大约：60 ms

6.1.2.3.11 Tick 第 4 块：更新 IMU/odometry/ fall 状态

6.1.2.3.12 Tick 第 5 块：读取 Intent Snapshot

6.1.2.3.13 Tick 第 6 块：Safety Gate

这里解决：“用户想让机器人怎么走”是否已经超时、是否应该被限制。

6.1.2.3.14 Tick 第 7 块：处理一次性 power request

6.1.2.3.15 Tick 第 8 块：处理 Skill / Sit / Ground Pick

`ground_pick`、`sit_toggle`、`skill requests` 这些请求不是直接产生 `joint target`。它们只是改变：Controller 的内部状态。例如：

`robot.do kick`



`controller.start_skill()`



`active = kick`

然后真正的 **target** 要到后面的: **controller.step()**才产生。这一点非常重要: **IPC request** 是“改变控制器状态”，不是“直接输出电机目标”。

6.1.2.3.16 Tick 第 9 块: 处理声音

6.1.2.3.17 Tick 第 10 块: 处理 ModeSwitch / Policy Reload

6.1.2.3.18 Tick 第 11 块: Shutdown/Battery Shutdown

6.1.2.3.19 Tick 第 12 块: Limp-Fall

6.1.2.3.20 Tick 第 13 块: 决定是否进入 driving
这是整个 **loop** 最重要的一个条件判断:

```
let driving =  
    snapshot.enabled  
    && bringup == Bringup::Ready  
    && controller.is_some()  
    && !in_limp_fall  
    && sensors.is_some()  
    && imu_warm  
    && !powered_off;
```

只有满足所有条件, **policy** 才真正拥有机器人。

6.1.2.3.21 Tick 第 14 块: 处理 driving 开始 / 停止

6.1.2.3.22 Tick 第 15 块: 电池电压补偿
目的是尽量保持 **servo** 的有效控制响应。

6.1.2.3.23 Tick 第 16 块: 生成 Joint Targets

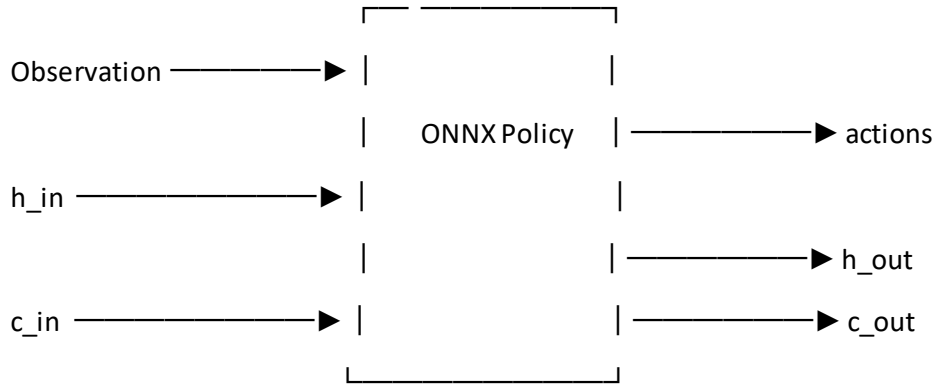
6.1.2.3.24 Tick 第 17 块: Controller.step()
在 **robotd** 的 **control loop** 里面, 当机器人处于真正 **driving** 状态时, 会进入:

```
controller.step(  
    sensors,  
    &command,  
    body_active,  
    period.as_secs_f64(),  
    scale_mult,
```

)

也就是说：`controlloop` 本身不直接调用 `ONNX`。它把传感器、`command` 等交给 `Controller`。而 `Controller` 才是控制算法这一层。`robotd/src/control.rs` 文件开头甚至直接写明：`Controller` 位于 `Robotlo::read` 和 `safety apply` 之间，只做计算，自己没有 `IO handle`，因此不能直接控制电机。

所以模型实际上是：



`Policy` 并不是直接学习“绝对关节角度”，而是学习相对于默认/home pose 的动作偏移。

6.1.2.3.24.1 *Controller 第 1 块：选择哪个 Network*

6.1.2.3.24.2 *Controller 第 2 块：构造 Observation*

6.1.2.3.24.3 *Controller 第 3 块：Action → Joint Target*

6.1.2.3.24.4 *Controller 第 4 块：Low-pass*

6.1.2.3.24.5 *Controller 第 5 块：更新自己的状态*

6.1.2.3.25 *Tick 第 18 块：Theremin / Chorale / Mouth*

6.1.2.3.26 *Tick 第 19 块：最终进入 Safety*

Policy

↓

targets

↓

表达层修改

↓

Safety.apply()



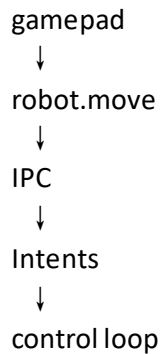
由 RobotIo 执行实际的 Dynamixel 写操作。因为是串口电机，因此实际是调用 Dynamixel 的串口输出。

6.1.2.3.27 Tick 第 20 块：发布 RobotState

6.1.2.3.28 Tick 第 21 块：记录 loop health

6.1.2.4 *robotd 的一些主要设计要点*

1. 启动后不会自动让机器人站起来。启动后，Duck 处于瘫软状态，为进入 update 准备。
2. 外部控制不是直接控制电机。比如键盘不是直接发指令给电机，而是：



即键盘发出机器人移动命令，通过 IPC（进程间通信）作为 intents 给控制循环（前面讲的控制线程），由控制线程控制电机。

3. Intents 是 IPC 和 control loop 之间的“缓冲层”

intents.rs 是非常重要的文件。里面不是 mutex^[3]，而是多个独立的 slot^[4]。

每一个 50 Hz tick，control loop 执行：

```
let snapshot = intents.snapshot();
```

一次性得到这一 tick 的输入。拿到 Intent 后：

```
let (gated, deadman) =
  safety.gate(snapshot.command, snapshot.twist_age);
```

Intent

▼

Safety.gate()

- twist 是否超时？
- 是否允许运动？
- 有哪些限制？

如果 gamepad 不再发送：robot.move

最后：

```
twist_age > deadman timeout
```

就会被 safetygate 截断。所以 gamepad 断线不会导致：

最后一次 vx=1.0

↓

机器人永远保持 vx=1.0

而会进入 deadman。

3 Mutex（互斥锁）是多线程编程中用于实现线程同步的核心机制，全称为 Mutual Exclusion（互斥），核心作用是保证同一时刻只有一个线程能访问共享资源，彻底避免多线程并发操作引发的数据竞争问题。

核心原理与作用

互斥访问：它就像一把带唯一钥匙的锁，线程在进入访问共享资源的临界区前必须先获取锁，操作完成后释放锁；其他试图获取已被占用锁的线程会被阻塞，直到锁被释放才能继续执行。

解决数据竞争：当多个线程同时读写同一块共享内存（如全局计数器、公共文件）时，没有同步机制会导致操作结果不可预测，mutex 通过强制序列化访问，保证共享数据操作的逻辑原子性，避免数据损坏。

严格所有权：互斥锁具有线程相关性，只有持有锁的线程才能释放它，跨线程释放锁会直接触发异常或程序错误。

不同编程语言和系统中提供了适配不同场景的 mutex 类型：

类型	核心特性	适用场景
普通互斥锁（std::mutex）	基础不可递归，重复加锁直接死锁	常规单重临界区保护，C++ 标准库默认类型
递归互斥锁（recursive_mutex）	允许同一线程多次加锁，需对应次数解锁 递归函数	嵌套调用中需要重复访问同一共享资源的场景
计时互斥锁（timed_mutex）	支持带超时的非阻塞加锁操作	避免线程无限期等待，防止死锁卡死

系统级命名互斥锁	跨进程可见，可同步不同进程中的线程	多进程共享全局资源的场景，如 Windows 系统全局同步
----------	-------------------	-------------------------------

基础使用规范：

1、标准流程遵循「加锁 → 操作共享资源 → 解锁」的模式，直接手动调用 `lock()/unlock()` 容易因异常、提前返回导致忘记解锁，引发死锁。

2、现代编程实践中优先使用 RAII 风格的自动锁管理类，如 C++ 的 `std::lock_guard`、Go 的 `defer mu.Unlock()`，在对象构造时自动加锁，析构时自动解锁，从语法层面避免遗漏解锁操作。

3、关键注意事项：

避免死锁：多个线程获取多把锁时，必须按全局统一的固定顺序申请锁，禁止循环等待。

控制锁粒度：只对真正需要保护的临界区代码加锁，不要在持有锁时执行耗时 I/O、网络请求等操作，避免大幅降低程序并发性能。

禁止在中断上下文使用 `mutex`：`mutex` 加锁可能导致线程休眠，中断上下文不允许休眠，否则会引发系统崩溃。

4 `slot` 在 Rust 官方文档与标准库实现中，线程相关的 `Slot`（插槽）并非单一独立概念，而是作为多线程/并发场景下的底层存储单元，分布在通道实现、Tokio 调度器、线程本地存储三大核心场景中，所有定义均来自 Rust 标准库源码与官方公开文档。

通道（Channel）中的消息 `Slot`

这是 Rust 标准库中定义最明确的 `Slot` 概念，是 `mpsc` 通道底层实现的核心存储单元，在标准库源码中直接以 `struct Slot<T>` 形式定义。

核心结构：每个 `Slot` 包含两个字段：

`msg: UnsafeCell<MaybeUninit<T>>`：用于未初始化状态下安全存储消息数据，避免不必要的内存初始化开销

`state: AtomicUsize`：原子状态标记位，用三个常量标记 `Slot` 生命周期：`WRITE(1)` 表示消息已写入、`READ(2)` 表示消息已被读取、`DESTROY(4)` 表示槽位正在销毁回收

无界通道的块式管理：无界通道采用链表块结构，每个 `Block<T>` 块内置 `BLOCK_CAP=31` 个 `Slot`，连续分配多个消息到同一块中，大幅降低内存分配频率、提升缓存命中率

无锁同步逻辑：发送线程通过原子操作标记 `WRITE` 位，接收线程自旋等待该标记后读取数据，全程无需互斥锁即可完成并发安全的消息传递

Tokio 调度器的 LIFO `Slot`

Tokio 多线程异步调度器中的专属优化组件，属于 Tokio 官方文档明确标注的调度特性，用于优化消息传递场景下的任务延迟。

核心作用：每个 `worker` 线程维护一个独立的专属 `Slot`，当任务 A 发送消息唤醒任务 B 时，任务 B 不会被推入任务队列尾部，而是直接存入当前线程的这个 `Slot` 中，下一次调度时优先执行任务 B，大幅降低消息传递的跨线程延迟

特性与配置：该 `Slot` 中的任务无法被其他工作线程窃取，当任务的 `poll` 执行时间较长时，可能导致整体吞吐量下降。官方提供 `disable_lifo_slot()` 配置方法可关闭该优化，将所有任务推入可被窃取的本地队列

-
4. 真正的 50 Hz tick 这就是核心。
每 20 ms: `ticker.tick().await`;
并且: `MissedTickBehavior::Skip`
不是 `Burst`, 也不是 `Delay`。因此如果某一次执行太慢:
tick 1
tick 2
tick 3 ← missed
tick 4
不会:
tick 3
tick 3
tick 3
补回来。
也不会因为一次延迟导致整个周期永久漂移。

6.2 ROBOTD 中关于 ONNX 的模型声明、例化、调用

`MicroDuck` 并不是 `robotd` 自己直接管理 `ONNX Runtime` 的全部细节。真正的 `ONNX` 模型加载、`session` 创建、`shape` 校验、`inference` 都已经抽到了 `duck-control::policy`, 即在 `control` 循环中; `robotd` 负责参数解析、策略选择和 50Hz 控制循环。

最核心的文件: `duck-control/src/policy.rs`。

6.2.1 定义: 定义一个 ONNX 模型对象

文件: ``duck-control/src/policy.rs``。定义阶段只是声明“我有一个 `ONNX` 模型对象”。还没有加载 ``onnx`` 文件, 也还不能执行。

6.2.2 创建: 把 ``onnx`` 文件加载成 ``Session``

还是在: ``duck-control/src/policy.rs``, 创建发生在 ``Network::open()`` 里面。

线程本地存储 (TLS) 中的数据 Slot

线程本地存储场景下, 每个独立线程为 `TLS` 变量分配的专属存储位置, 是 `Rust` 标准库 `std::thread::LocalKey` 与第三方 `thread_local` 库的底层实现细节。

标准库实现: `LocalKey` 内部通过间接跳转 `thunk` 函数, 为每个线程动态分配专属 `Slot` 存储 `TLS` 数据, 保证每个线程访问到完全独立的副本, 无需任何同步操作

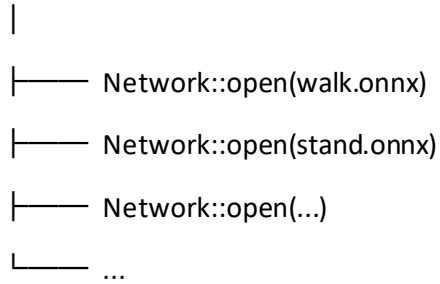
第三方库实现: `thread_local` 库通过分层原子指针桶结构, 为每个线程分配唯一的 `Slot` 索引, 实现无锁的 `TLS` 数据访问, 性能优于系统原生 `pthreadTLS` 接口

生命周期规则: 每个线程首次访问 `TLS` 变量时, 该线程对应的 `Slot` 会自动执行惰性初始化; 线程退出时, `Slot` 内的数据自动执行 `Drop` 析构

6.2.3 `robotd` 创建它

`Policy::load()` 内部:

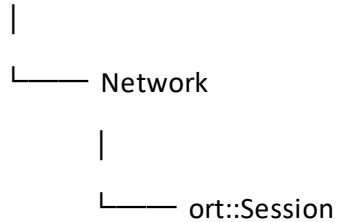
`Policy::load()`



`robotd`



`Policy`



对于 `walking`:

`alpha_walking.onnx`



`Network`



`Session`

6.2.4 执行: 调用 ONNX

真正执行发生在:

``duck-control/src/policy.rs``

``Network::run()``。

核心可以简化成:

```
let outputs = self.session.run(inputs)?;
```

这就是最关键的一句。

也就是说：

```
```text
```

```
observation
```

```
|
```



```
ONNX input tensor
```

```
|
```



```
self.session.run(...)
```

```
|
```



```
ONNX Runtime
```

```
|
```



```
[1, 14]```
```

#### 6.2.5 `robotd` 是怎么调用它的？

在控制循环中，`robotd` 最终会走到：

```
self.policy.infer(&observation, net)
```

然后：

```
Controller::step()
```

```
|
```



```
Policy::infer()
```

```
|
```



```
Network::run()
```

---

|  
▼  
session.run()

|  
▼  
ONNX

|  
▼  
14 actions

所以你只需要记住这条调用链：

robotd

|  
▼  
Policy::infer()

|  
▼  
Network::run()

|  
▼  
Session::run()

|  
▼  
alpha\_walking.onnx

## 6.2.6 三个动作总结

### ① 定义

告诉 Rust：

```
struct Network{
 session: Session,
```

---

```
}
```

意思：一个 **Network** 里面有一个 **ONNX Runtime Session**。

## ② 创建

```
let session = Session::builder()
 .commit_from_file(path)?;
```

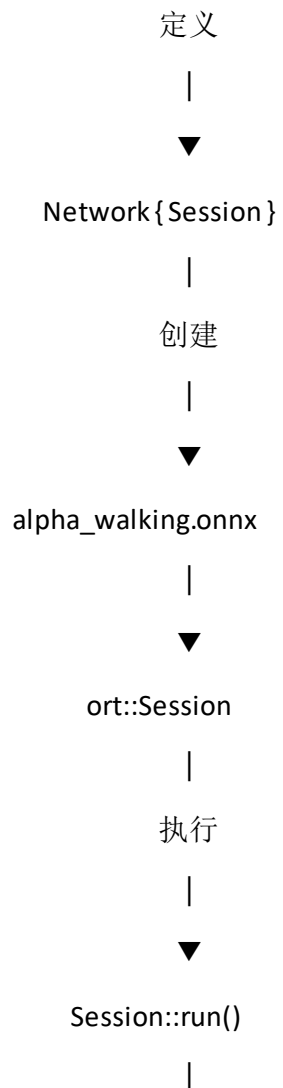
意思：把 ``alpha_walking.onnx`` 加载进 **ONNX Runtime**，创建 ``Session``。

## ③ 执行

```
let outputs = self.session.run(inputs)?;
```

意思：把 61 维 **observation** 喂给 **ONNX**，执行神经网络，得到 14 维 **action**。

最简单的一张图





`[1, 61] → [1, 14]`

定义 声明对象长什么样。

创建 = 加载 ONNX 文件，得到 ``Session``。 \*\*

执行 = ``Session::run()``，输入 `observation`，输出 `action`。 \*\*

这三个层次分清楚后，再看 ``robotd`` 的其它代码就不会乱了。

## 6.3 从 ONNX 文件到程序输出：完整流程

这段讲述一般的 ONNX 概念和工作流程。

### 6.3.1 先理解：ONNX 文件本身是什么

ONNX（Open Neural Network Exchange）文件本质上是一个“静态计算图描述”——它只描述“数据怎么流动、经过哪些算子”，不包含执行引擎本身。所以一个“.onnx”文件在程序里是“跑不起来的”，而必须：

ONNX 模型文件 → 推理引擎（Runtime） → 你的程序拿到输出

这正是 MicroDuck 架构里分层的原因：策略训练在 ``microduck_rl``（PyTorch + PPO）里完成（后面的章节讲述），导出成 ONNX 后，MicroDuck 机器人上的 ``robotd`` 守护进程用 ONNX Runtime 加载它以 50Hz 实时推理——训练框架和推理引擎完全解耦。MicroDuck 控制循环就是`状态→观测→神经网络(ONNX)→动作→物理→新状态`的闭环。

### 6.3.2 完整过程（四步）

第 1 步：确认“模型契约”（最容易被忽略的一步）

拿到 ONNX 文件第一件事不是写代码，而是搞清楚它的输入输出契约——形状、数据类型、语义。以 MicroDuck 的行走策略为例：

---

-输入: `obs`<sup>[5]</sup>, float32, shape<sup>[6]</sup>`[1, 61]`

-输出: `actions`, float32, shape `[1, 14]` (对应 14 个舵机的动作)

-控制频率: 50Hz (每 20ms 推理一次)

-重要细节: 观测归一化器已经烘焙进了 ONNX 图内部, 调用方不要再自己归一化。

查看方法 (任选其一):

```
``python
import onnx

model = onnx.load("policy.onnx")

print(onnx.helper.printable_graph(model.graph))
``
```

或在线工具 [netron.app](https://netron.app) 可视化打开。

## 第 2 步: 安装推理引擎

最常见的选择是 ONNX Runtime (微软维护, 支持 Python/C++/C#/Rust/Java, 可在 CPU 上零依赖运行; RK3566/RDK X5 这类 ARM64 板上也有对应预编译包:

```
``bash

pip install onnxruntime
```

---

5 `obs`=observation (观测值) 的缩写。这是强化学习、机器人、自动驾驶里最常见的<sup>变量名</sup>之一。ONNX 导出的强化学习策略模型很多都把输入节点命名成 `obs`。举个四足机器人例子, Unitree、Isaac Lab、Isaac Gym 里很常见:

```
obs =[身体角速度 3 维
重力方向 3 维
12 个关节位置
12 个关节速度
上一步动作
目标速度]
```

6 在 ONNX、Tensor、深度学习里, Shape (形状) 是最最重要的概念之一。可以简单理解为:

Shape 描述了一个张量 (Tensor) 每个维度有多大, 类似于矩阵的维数。如: `input:float32[1,3,224,224]`, 这里面 shape 就是 `[1,3,224,224]`。`obs`, float32, shape `[1, 61]` 的意思就是: `obs` 变量是 float32 类型的 61 维向量。

---

```
如果有 NVIDIA GPU: pip install onnxruntime-gpu
```

```
...
```

备选引擎（按场景）：TensorRT（英伟达加速）、RKNN（瑞芯微 NPU）、ONNX Runtime Web（浏览器，MicroDuck 官方沙盒就是这么做的）。

第 3 步：加载模型，跑一次推理

Python 完整最小示例（对应 MicroDuck 契约）：

```
```python
```

```
import numpy as np
```

```
import onnxruntime as ort
```

①创建推理会话（Session 是资源句柄，只创建一次，别放进循环里）

```
sess = ort.InferenceSession("policy.onnx", providers=["CPUExecutionProvider"])
```

② 确认输入输出（与模型契约核对）

```
inp = sess.get_inputs()[0]
```

```
print(inp.name, inp.shape, inp.type) # 例如: obs [1, 61] tensor(float)
```

③ 准备输入：严格按契约构造 float32 数组

```
obs = np.zeros((1, 61), dtype=np.float32) # 实际应从 IMU/舵机读取观测
```

④ 推理：传入 {输入名: 数组}，拿到输出列表

```
actions = sess.run(None, {inp.name: obs})[0]
```

```
print(actions.shape) # (1, 14)
```

```
...
```

C++ 等价写法（真机 `robotd` 的 Rust 逻辑本质相同）：

```
```cpp
```

```
#include <onnxruntime_cxx_api.h>
```

```
Ort::Env env(ORT_LOGGING_LEVEL_WARNING, "demo");
```

```
Ort::Session session(env, L"policy.onnx", Ort::SessionOptions{});
```

```
std::vector<int64_t> dims{1, 61};
```

```
std::vector<float> obs(61, 0.0f);
```

---

```

auto mem = Ort::MemoryInfo::CreateCpu(OrtArenaAllocator, OrtMemTypeDefault);

Ort::Value input = Ort::Value::CreateTensor<float>(mem, obs.data(), obs.size(), dims.data(),
dims.size());

auto outputs = session.Run(Ort::RunOptions{nullptr},

 session.GetInputNames().data(), &input, 1,

 session.GetOutputNames().data(), 1);

float* actions = outputs[0].GetTensorMutableData<float>();
...

```

Rust 中可用 `ort` crate；这也是 MicroDuck 官方栈在用的技术路线。

#### 第 4 步：嵌入控制循环

单次 `sess.run` 拿到的是原始张量；在机器人程序里，你要把它放进实时循环：

```

```python
import time

RATE_HZ = 50                                # 与训练时的控制频率严格对齐！
dt = 1.0 / RATE_HZ

while True:
    t0 = time.time()

    obs = read_sensors()                     # ① 从 IMU/舵机编码器组 61 维观测

    actions = sess.run(None, {"obs": obs})[0] # ② ONNX 推理 → 14 维动作
    send_to_servos(actions[0])                # ③ 动作下发到舵机总线
    time.sleep(max(0, dt - (time.time() - t0))) # ④ 补足到 50Hz
...

```

工程要点：

1. 频率对齐：训练时 50Hz 训出的策略，部署时也必须 50Hz 调用，否则观测的时间语义错位，策略会失衡。

2. 预处理/后处理与模型约定一致：比如 MicroDuck 的归一化已烘焙进图，就别重复做；如果导出时没烘焙，就要在代码里手动做。官方导出器 `scripts/export.py` 会自动烘焙归一化器，这是正确行为所必需的。

3. 会话只建一次，循环里只调用 `run()`；每次新建 Session 的开销在实时控制中不可接受。

7 MICRODUCK 的训练

地址：https://github.com/pollen-robotics/microduck_rl

入手点：

- README.md：掌握「仿真→训练→部署」全流程（基于 MuJoCo Warp 仿真、PPO 算法）；
- src/agents/：强化学习智能体实现（reward 设计、策略网络结构）；
- scripts/export.py：模型导出为 ONNX 格式的脚本（连接仿真与硬件的关键）。

核心闭环：Sim2Real 链路（架构设计核心）

这是 Microduck 软件架构的灵魂，也是开源的核心价值：

1. 仿真训练：在 MuJoCo Warp 中搭建机器人模型，通过 PPO 算法训练行为策略（如行走、跌倒恢复）；
2. 模型导出：将训练好的策略导出为 ONNX 格式（microduck_rl/scripts/export.py）；
3. 硬件部署：机器人通过 ONNX Runtime 加载模型，在 50Hz 控制环中执行；
4. 迭代优化：通过传感器数据观察硬件与仿真的差异，调整仿真参数（如摩擦、延迟）后重新训练。

8 浏览器仿真工具（可视化理解架构运行）

无需本地部署，直接在浏览器体验架构闭环：

地址：<https://huggingface.co/spaces/pollen-robotics/microduck-simulator>

入手点：

- 操作交互：用 WASD / 游戏手柄控制机器人（行走、踢球、轮滑），直观感受「输入指令→策略推理→电机执行」的链路；

-
- 核心观察：切换「腿部 / 轮滑模式」（按 M 键），理解不同 ONNX 策略如何被加载和执行；
 - 调试模式：添加 URL 参数 ?boot=1 进入 BIOS 控制台，查看模型加载、策略初始化的实时日志（对应硬件端的启动流程）。

9 官方文档与特性说明（补充架构设计理念）

- 从 GitHub 仓库的 docs/ 目录入手，重点看 agents.md（强化学习策略设计手册）；
- 参考产品页面技术说明：<https://pollen-robotics.com/microduck/>（明确软件与硬件的交互边界，如传感器数据格式、电机控制协议）。

10 RL 算法
